

Вариант А

Име: _____ ФН: _____ група: _____

Лексикално дърво ще наричаме двоично дърво за търсене, в което всеки възел пази стойност малка буква от латинската азбука. За стойностите във възлите са изпълнени правилата за дърво за търсене.

Обединение на лексикални дървета T1 и T2 наричаме множеството от всички стойности на възли, които се срещат в дървото T1 или в дървото T2. Сечение на лексикалните дървета T1 и T2 наричаме множеството от всички стойности на възли, които се срещат в дървото T1 и в дървото T2. Разлика на лексикалните дървета T1 и T2 наричаме множеството от всички стойности на възли, които се срещат в дървото T1 и не се срещат в дървото T2. Дърво е под-дърво на друго, ако при премахване на някои върхове от едното дърво ще се получи другото, без пренареждане на останалите.

Реализирайте клас, представящ лексикално дърво, който поддържа следните функции:

- **void build(const char* letters);** // построява лексикално дърво от символите в подадения низ. Изисква се дървото да е идеално балансирано. За тази функционалност се допуска сложност по-голяма от линейна по броя букви.
- **Tree union(const Tree& right) const;** // връща дърво - обединение на текущото и подаденото.
- **Tree intersect(const Tree& right) const;** // връща дърво - сечение на текущото и подаденото.
- **Tree diff(const Tree& right) const;** // връща дърво - разлика на текущото и подаденото.
- **bool subTree(const Tree& right) const;** // проверява дали подаденото дърво е поддърво на текущото.
- **static bool check(const Tree& right);** // Проверява дали подаденото дърво е валидно.
- **bool balanced() const;** // Проверява дали текущото дърво е балансирано по височина.
- Необходимите методи за поддържане на коректен жизнен цикъл на обектите
- Оператор за извеждане в поток, който извежда дървото по следния начин:
{<key> {<left-sub-tree> {right-sub-tree}}, където празно дърво изобразяваме с {}

- Сложността на всички операции трябва да е линейна по общия брой елементи в дърветата, ако не е посочено друго.
- Изисква се пълна гаранция при възникване на изключения. Допуска се използване на допълнителна памет, пропорционална на височината на дърветата.
- От структурите данни в STL се позволява използването на `std::stack` и `std::queue`. Не се позволяват други структури от данни и алгоритми. Позволено е използването на типовете и класовете за работа с потоци и изключения, както и библиотеката за работа със символни низове `cstring`.

Вариант Б

Име: _____ ФН: _____ група: _____

Сигнално дърво ще наричаме двоично дърво за търсене (BST), в което всеки възел пази положителна, целочислена стойност (сигнал).

Линеен път в дървото наричаме всяка верига от възли, в която всеки родител има **точно едно дете**. **Компресия** на сигнално дърво наричаме процес, при който всички възли, участващи в линейни пътища (имащи само по едно дете), биват премахнати, като връзката се прехвърля директно към единственото им дете. Неговата стойност не се променя. Тази операция се извършва дотогава, докато в дървото останат само листа или възли с по две деца.

Сгъване на дърво наричаме операция, при която лявото поддърво на корена се "влива" в дясното. При сгъването, ако дадена структурна позиция съществува и в двете поддървета, стойностите на възлите се сумират в дясното, а левият възел се унищожава. Ако позицията съществува само в лявото, възелът се премества в дясното. След операцията лявото поддърво на корена е празно.

Сливане на две дървета е операция, при която две дървета се "наслагват" едно върху друго - ако на дадено място и двете имат елемент, то полученото дърво има нов елемент със стойност сумата от двата възела. Ако едно от дърветата няма съответен елемент - в резултата се добавя такъв със същата стойност.

Реализирайте клас, представящ сигнално дърво, за който трябва да се реализират следните методи:

- **void build(istream& stream);** // построява дърво по данни, намиращи се в подадения поток. Форматът на входа е {<key> {<left-sub-tree> {<right-sub-tree>}}, където празно дърво изобразяваме с {}. Проверете коректност на входа!
 - **void compress();** // компресира текущото дърво.
 - **void fold(treeNode* root);** // сгъва поддървото с корен root.
 - **Tree merge(const Tree& right) const;** // слива подаденото дърво с текущото.
 - **static int isBalanced(const Tree& tree);** // връща 0 ако дървото не е балансирано, 1 ако е балансирано по височина и 3 ако е балансирано по тегло
 - **static bool check(const Tree& tree);** // Проверява дали подаденото дърво е валидно сигнално дърво.
 - **static bool isCompressed(const Tree& tree);** // Проверява дали подаденото дърво е компресирано.
 - Необходимите методи за поддържане на коректен жизнен цикъл на обектите.
 - Оператор за извеждане в поток, който извежда стойностите на елементите на дървото в строго намаляващ ред. Ако има еднакви стойности те трябва да пропуснат.
- Сложността на всички операции трябва да е линейна по общия брой елементи в дърветата.
 - Изисква се пълна гаранция при възникване на изключения. Допуска се използване на допълнителна памет, пропорционална на височината на дърветата.
 - От структурите данни в STL се позволява използването на std::string, std::stack и std::queue. Не се позволяват други структури от данни и алгоритми. Позволено е използването на типовете и класовете за работа с потоци и изключения.